

Mini Payment Router Simulator

September 16, 2026

Contents

1	App Architecture (Frontend–Backend Interaction)	2
2	Key Code Components and Their Purpose	4
2.1	Packages inside <code>payment-router</code>	4
2.2	The five services and what each one owns	4
2.3	Strategy Pattern — isolates DFSP-specific behaviour	4
2.4	Adapter Pattern — hides DFSP API differences	4
3	How the API Works Internally	6
3.1	Endpoints	6
3.2	Quote	6
3.3	Transfer	6
3.4	Error response (all endpoints)	7
3.5	Validation	7
4	Docker Setup Explanation	8
5	Additional Notes: Design Decisions and Testing	9
5.1	Notable design decisions	9
5.2	Tests	9

1 App Architecture (Frontend–Backend Interaction)

The system is five containers wired together by Docker Compose: a React frontend, the main `payment-router` backend, two dummy Digital Financial Service Providers (`dfsp-a`, `dfsp-b`), and a PostgreSQL database.

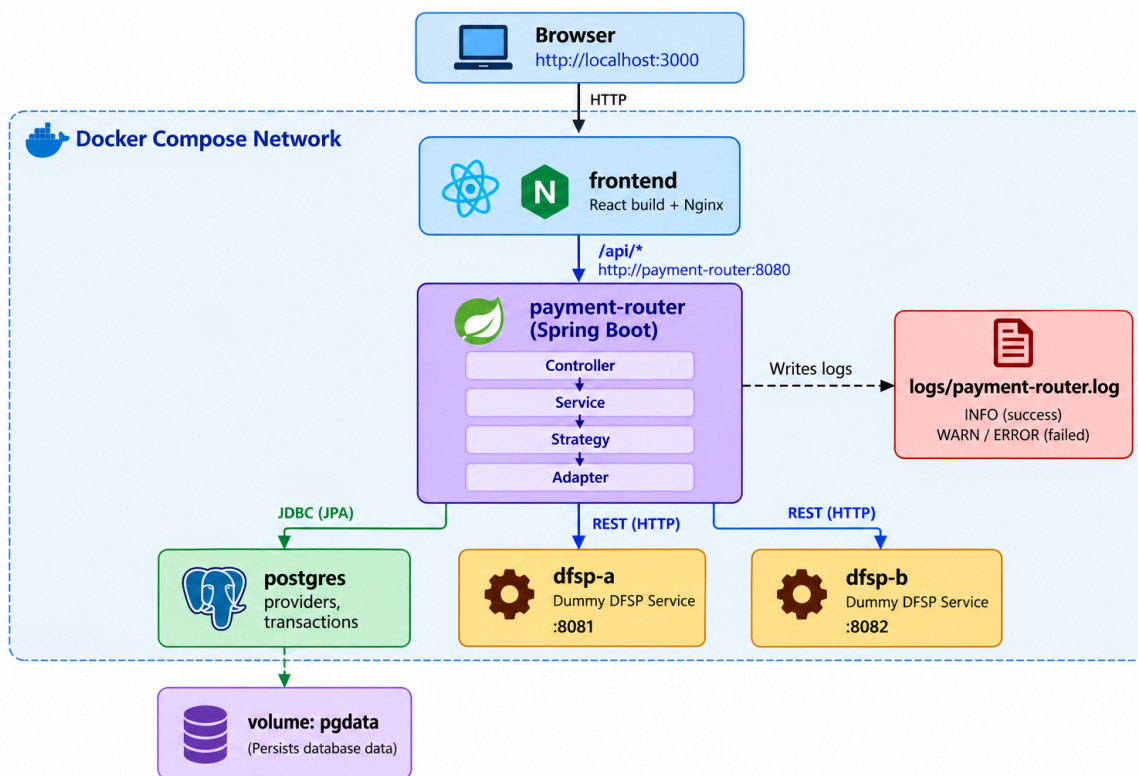


Figure 1: Browser to frontend (React + Nginx), to payment-router (Controller → Service → Strategy → Adapter), to postgres, dfsp-a and dfsp-b, all inside one Docker Compose network.

How the frontend talks to the backend

- The browser talks **only** to **frontend**. In Docker, **Nginx** serves the built React static files and reverse-proxies every `/api/*` request to `payment-router:8080`. In local development, the **Vite dev server** proxies `/api` to `http://localhost:8080` the same way. Either way, the browser sees a single origin, so CORS is never involved in the normal path.
- **Only payment-router** talks to **postgres**, **dfsp-a** and **dfsp-b**. The two DFSPs never talk to each other or to the database — every cross-service call goes through the router.
- Containers reach each other by Docker Compose **service name** (`postgres:5432`, `http://dfsp-a:8081`), never by `localhost`.
- A proxy must forward the browser's original **Host** header (Nginx: `proxy_set_header Host $http_host`). If it did not, Spring would see a mismatched Origin/Host on a POST and reject it as a cross-origin request.

Runtime interaction, step by step

1. The frontend loads on `http://localhost:3000` and calls `GET /api/providers` to populate the source/destination dropdowns — nothing is hard-coded in the UI.
2. The user picks a source, a destination and an amount, then clicks **Get quote**. The frontend calls `POST /api/quotes`. Nothing is sent to a DFSP and nothing is saved; it is a pure calculation.
3. The user clicks **Confirm transfer**. The frontend calls `POST /api/transfers` with the same three fields. This time the router actually calls the destination DFSP over HTTP, waits for its answer, and saves exactly one row in PostgreSQL — `SUCCESS` or `FAILED`, either way.
4. The frontend renders the result card returned by that call: status badge, transaction ID, the DFSP's own message, and the pricing snapshot. The frontend holds no business logic of its own — every number shown came from the backend response.

Layering inside payment-router

```
Controller -> Service -> DfspStrategy -> DfspClient (adapter) -> DFSP over HTTP
      |
      +-----> Repository -> PostgreSQL
```

2 Key Code Components and Their Purpose

2.1 Packages inside `payment-router`

Package	Responsibility
<code>controller</code>	HTTP endpoints; triggers <code>@Valid</code> ; delegates to services
<code>service</code>	Business validation, quote calculation, transfer orchestration; picks the strategy matching the destination provider's code
<code>strategy</code>	One strategy per DFSP (pricing + which adapter to call)
<code>client</code>	Adapters translating to and from each DFSP's own API
<code>entity,</code> <code>repository</code>	JPA entities and Spring Data repositories
<code>exception</code>	Custom exceptions and the global error handler
<code>config</code>	HTTP client timeouts, CORS, provider seed data

2.2 The five services and what each one owns

- **Frontend** — a form for source/destination/amount; shows the quote and the transfer result; contains no business logic.
- **Payment Router** — validates requests, calculates quotes, routes transfers via the right strategy/adapter, saves every attempt, and logs events. This is the only "smart" component in the system.
- **DFSP-A** — dummy provider, its own API format, rejects transfers above 50,000.00 taka.
- **DFSP-B** — dummy provider, a deliberately different API format (paisa, different field names), rejects transfers above 25,000.00 taka.
- **PostgreSQL** — stores providers and transactions only; reachable only by `payment-router`.

2.3 Strategy Pattern — isolates DFSP-specific behaviour

The interface `DfspStrategy` declares three operations: `getProviderCode()`, `calculateQuote(amount, destinationProvider)` and `executeTransfer(request, destinationProvider)`. `DfspAStrategy` and `DfspBStrategy` each implement it. `calculateQuote` has one common default implementation on the interface itself ($\text{fee} = \text{amount} \times \text{fee_percentage} / 100$, rounded `HALF_UP`; $\text{total} = \text{amount} + \text{fee}$), since both DFSPs currently price the same way; a DFSP whose pricing ever differs would simply override that one method.

Each strategy's `executeTransfer` routes to its own `DfspClient` adapter (`DfspAStrategy` → `DfspAAdapter`, `DfspBStrategy` → `DfspBAdapter`). `TransferService` and `QuoteService` never write `if (code.equals("DFSP_A")) ...`; they ask "give me the strategy for this provider code" (a `stream().filter()` lookup over the injected list of strategies) and call it. Adding a third DFSP means adding one new strategy class, not editing existing logic.

2.4 Adapter Pattern — hides DFSP API differences

`DfspTransferRequest` and `DfspTransferResult` are the router's common, DFSP-neutral models. The interface `DfspClient` declares a single method, `transfer(baseUrl, DfspTransferRequest)`, implemented by `DfspAAdapter` and `DfspBAdapter`.

Adapter	Translates request to	Translates response from
DfspAAdapter	Decimal taka, DFSP-A's own field names	"SUCCESS"/"FAILED" → common SUCCESS/FAILED
DfspBAdapter	Integer paisa, DFSP-B's own field names	"ACCEPTED"/"REJECTED" → common SUCCESS/FAILED

Each adapter sends its DFSP-specific JSON to the provider's `base_url` (read from the `providers` table) and converts the reply back into the common model. Network errors (DFSP down, timeout) become a `DfspCommunicationException`, which the transfer service turns into a `FAILED` transaction rather than an HTTP error. Timeouts are configurable: `dfsp.client.connect-timeout=3s`, `dfsp.client.read-timeout=5s`.

3 How the API Works Internally

Every request follows the same internal path before any business logic runs: **Controller** receives the HTTP call, Spring's **@Valid** annotation triggers field-level validation on the request DTO first, and only a structurally valid request reaches **PaymentRequestValidator** for business-level checks (does the provider exist, is it active). Only after both checks pass does the **Service** layer pick a **DfspStrategy** and, for a transfer, call the matching **DfspClient** adapter and save the outcome.

3.1 Endpoints

Method	Path	Purpose
POST	/api/quotes	Calculate a quote (nothing is saved)
POST	/api/transfers	Execute a transfer (a transaction is saved)
GET	/api/providers	List providers (UI dropdowns)
GET	/api/status	Service status (used by the Docker healthcheck)

3.2 Quote

POST /api/quotes

```
{ "sourceProviderCode": "DFSP_A", "destinationProviderCode": "DFSP_B", "amount": 1000.00
}
```

200 OK

```
{
  "sourceProviderCode": "DFSP_A",
  "destinationProviderCode": "DFSP_B",
  "amount": 1000.00,
  "feePercentage": 1.50,
  "feeAmount": 15.00,
  "totalAmount": 1015.00
}
```

3.3 Transfer

POST /api/transfers has the same request body as a quote.

201 Created. A transaction row is created for SUCCESS and for FAILED; the business outcome is in **status**.

```
{
  "transactionId": "3f6e2c1a-8b0d-4c5e-9a4f-1d2b3c4d5e6f",
  "sourceProviderCode": "DFSP_A",
  "destinationProviderCode": "DFSP_B",
  "amount": 1000.00,
  "feePercentage": 1.50,
  "feeAmount": 15.00,
  "totalAmount": 1015.00,
  "status": "SUCCESS",
  "message": "ACCEPTED (ref B-PAY-000123)",
}
```

```

    "createdAt": "2026-09-14T10:01:12Z"
  }

```

3.4 Error response (all endpoints)

```

{
  "timestamp": "2026-09-14T10:01:12Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Provider not found: DFSP_X",
  "fieldErrors": {}
}

```

HTTP	When
400	Field validation failed, malformed JSON, provider not found or inactive
404	No endpoint for the URL
405	HTTP method not supported by the endpoint
415	Body not sent as <code>application/json</code>
500	Unexpected server error (details only in the log)

All of these are produced by `GlobalExceptionHandler` with the same JSON shape. A DFSP timeout gets its own FAILED message: “Destination DFSP did not respond in time”. DFSP rejection or DFSP unavailability is **not** an HTTP error for the client — it is saved and returned as a FAILED transaction.

3.5 Validation

Rule	Where	Mechanism
Amount is required and greater than zero	Request DTO	<code>@NotNull</code> , <code>@Positive</code> + <code>@Valid</code>
At most 9 integer digits, 2 decimal places	Request DTO	<code>@Digits(integer = 9, fraction = 2)</code>
Source provider is required	Request DTO	<code>@NotBlank</code>
Destination provider is required	Request DTO	<code>@NotBlank</code>
Provider exists and is ACTIVE	<code>PaymentRequestValidator</code>	DB lookup → 400

Source and destination are **allowed to be the same provider**. A DFSP sending a payment to itself (e.g. `DFSP_A` → `DFSP_A`) is validated, priced and routed exactly like any other transfer, using that provider’s own strategy and adapter.

`GlobalExceptionHandler` (`@RestControllerAdvice`) converts `MethodArgumentNotValidException` (field errors) and `InvalidPaymentRequestException` (business rules) into the error JSON above. Quote and transfer share the same business checks through `PaymentRequestValidator`. The 9-digit limit keeps `amount` + `fee` inside the `NUMERIC(12,2)` columns, so a very large amount is rejected with 400 instead of failing when the transaction is saved.

4 Docker Setup Explanation

Service	Image / build	Port (host:container)	Depends on
frontend	node:22-alpine build → nginx:1.27-alpine	\${FRONTEND_ PORT:-3000}:80	payment-router (healthy)
payment-router	maven:3.9-eclipse-temurin-21 build → eclipse-temurin:21-jre-alpine	8080:8080	postgres (healthy), dfsp-a, dfsp-b
dfsp-a	Maven build → eclipse-temurin:21-jre-alpine	8081:8081	—
dfsp-b	Maven build → eclipse-temurin:21-jre-alpine	8082:8082	—
postgres	postgres:16-alpine	not published	—

- **Multi-stage Dockerfiles.** Build tools (Maven, Node) stay in the build stage; the final image only contains what is needed to run.
- **Healthcheck on postgres** (`pg_isready`). `payment-router` waits for `condition: service_healthy`, so it does not start before the DB accepts connections.
- **Healthcheck on payment-router** (`wget http://localhost:8080/api/status` inside its own container). `frontend` waits until the router is healthy.
- **Tests are not run inside the image build** (`-DskipTests`), because the router's tests need PostgreSQL. Run them with `./mvnw test` before building.
- **Nginx** (`frontend/nginx.conf`) serves the React build and proxies `/api/` to `http://payment-router:8080` with `Host $http_host`.
- `.env.example` lists the optional overrides (`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `FRONTEND_PORT`). Every value has a default in `docker-compose.yml`.
- **Named volume `pgdata`** keeps database data across restarts.
- **Bind mount** `./logs:/app/logs` makes the log file visible on the host.
- **Environment variables** configure the DB connection (`SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/payment_router`, `username`, `password`).
- DFSP addresses come from `providers.base_url`. In Docker they are seeded as service names through `DFSP_A_BASE_URL=http://dfsp-a:8081` and `DFSP_B_BASE_URL=http://dfsp-b:8082`.
- Start everything with `docker compose up -build`, then open `http://localhost:3000`.

5 Additional Notes: Design Decisions and Testing

5.1 Notable design decisions

Decision	Why
Fee priced from the destination provider, not the source	Whoever receives and processes a payment sets its price — mirrors how real inter-DFSP settlement works
Pricing snapshot stored on every transaction	A fee changed tomorrow must never rewrite what a transfer cost yesterday
Every valid attempt is saved, success or failure	A rejected, timed-out or unreachable transfer is still a business event, not an error to discard
Database write happens <i>after</i> the DFSP call, never wrapping it	A slow or hanging DFSP must never hold a database connection open
Two DFSP APIs made deliberately incompatible	Gives the Adapter Pattern a genuine reason to exist — taka vs paisa, SUCCESS/FAILED vs ACCEPTED/REJECTED
Strategy selection is a <code>stream().filter()</code> , no factory class	Only two DFSPs; a dedicated lookup class is pure overhead. Adding DFSP-C means one new strategy + adapter bean, nothing else changes
Source and destination may be the same provider	A DFSP paying itself is still a valid transfer; there's no reason to special-case it
<code>spring.jpa.open-in-view=false</code>	Forces every database read to finish inside the service layer; a stray lazy-load reached from a controller fails loudly instead of silently
Nginx forwards the browser's original Host header	Otherwise Spring sees a mismatched Origin/Host and rejects the POST as cross-origin
<code>BigDecimal</code> / <code>NUMERIC</code> for every amount	Binary floating point cannot represent 15.00 exactly, and a fee is not an approximation
No quotes table	A quote is a calculation, not a record — nothing to persist
No authentication	Out of scope for this assessment

5.2 Tests

```
cd backend/payment-router && ./mvnw test # 56 tests
cd backend/dfsp-a && ./mvnw test # 5 tests
cd backend/dfsp-b && ./mvnw test # 5 tests
```

The router's tests run against a **real PostgreSQL** on port 5433, not an in-memory database — `NUMERIC` precision and constraint behaviour need to be exercised as they actually behave. The adapter tests (`DfspAAdapterTest`, `DfspBAdapterTest`) start a tiny real local HTTP server standing in for the DFSP (the JDK's own `HttpServer`, no test framework involved), so the JSON the adapter actually sends and the response it actually parses are both exercised over a real socket.

```
docker compose up --build -d
bash scripts/smoke-test.sh # 21 checks against the real Compose stack, through Nginx
```

The smoke test is the only place the full request → validation → strategy → adapter → DFSP → database → response chain is exercised end to end, against the real stack — deliberately the one place this project uses a real HTTP client instead of an in-process HTTP-layer testing tool. There is no frontend test suite.